Reproducible Quantum ESPRESSO Workflows Across macOS and Linux

Shahriar Pollob* M. Shahnoor Rahman†

*Mentee †Mentor

Abstract

We report the successful porting and validation of Quantum ESPRESSO (v7.4.1) on the Apple M4 architecture under macOS 15 (Sequoia). Addressing significant toolchain regressions in the latest Xcode Command Line Tools, we established a reproducible compilation workflow using a hybrid Homebrew/GCC environment. The native build was validated against standard silicon electronic structure and aluminium equation-of-state tutorials. Furthermore, we conducted a comparative performance benchmark against a standard Linux workstation (AMD Ryzen 5600G + NVIDIA RTX 4090). Analysis reveals that for the small-scale workloads ($N_{atoms} < 10$) typical of educational training, the Mac mini M4 minimizes workflow latency, outperforming the GPU-accelerated server which incurs significant initialization overhead for small systems. All build scripts, validation data, and provenance artifacts are archived in public repositories to facilitate reproducibility.

Acknowledgements

I am grateful to the ICTP PWF: Physics for Bangladesh[1] organisers for making this online internship possible and for coordinating the remote collaboration. I would also like to thank M. Shahnoor Rahman[2] for his guidance, computing resources, timely feedback, and technical support throughout the project.

[1] https://indico.ictp.it/event/11042

[2] https://sites.google.com/view/mshahnoorahman

Mentor Approval

This page is reserved for mentor endorsement of the internship report and the activities documented herein.

Mentor: M. Shahnoor Rahman **Role:** Mentor

Statement: I have reviewed this report and approve it as an accurate record of the work completed during the ICTP PWF: Physics for Bangladesh internship.

Signature Date

Contents

1 Introduction

Quantum ESPRESSO (QE) is a standard open-source suite for quantum materials modeling. While Linux clusters remain the standard for production runs, the emergence of Apple Silicon (M-series) offers a compelling platform for local development and post-processing due to its high-bandwidth unified memory architecture. However, the recent release of macOS 15 introduced stricter preprocessor and linker defaults that broke existing build recipes for Fortran-based scientific codes.

This project focuses on two primary objectives: (1) Re-establishing a stable, reproducible build pipeline for QE 7.4.1 on the Mac mini M4, and (2) Quantifying its performance utility relative to a GPU-accelerated Linux server.

Section 2 details the resolution of architecture-specific build failures, including BLAS symbol mismatches and preprocessor flags. Sections 3 and 5 document the validation of the "Path A" (Silicon bands) and "Path B" (Aluminium EOS) workflows on both platforms. Finally, Section 6 analyzes the performance data, distinguishing between the high-throughput capabilities of the server and the low-latency advantages of the M4 chip for interactive educational tasks.

2 Porting Quantum ESPRESSO to macOS 15 on a Mac mini M4

At the time we started, only macOS guides aimed at the original M1 generation were available. Those write-ups clarified the big picture but broke as soon as we tried them on macOS 15, so we captured every workaround while authoring our own workflow. This chapter documents that journey of running QE 7.4.1 natively on a Mac mini M4 under macOS 15 (Sequoia). Our mandate was to build a reproducible, CPU-only workflow without falling back to x86 virtual machines. The accompanying repository[3] archives scripts, logs, and analysis artefacts that mirror the steps reported here.

[3] https://github.com/shahpoll/qe_macos15_build

2.1 Baseline objectives and constraints

We began by declaring three non-negotiable outcomes: (i) compile QE with full MPI/OpenMP support using Apple Silicon compilers, (ii) validate the toolchain by reproducing the silicon SCF–NSCF–bands workflow that underpins our convergence studies, and (iii) capture every workaround in automated scripts so a clean macOS 15 install could replay the configuration in one sitting. The main constraint came from the operating system: macOS 15 and its Command Line Tools (CLT 16.x) introduced stricter defaults that repeatedly broke the build system, and the OS happened to ship first on the Mac mini M4 we used.

2.2 Provisioning the native toolchain

Homebrew served as the package manager of record. Running `brew bundle` against the repository's `Brewfile` installed `gcc@14`, `open-mpi`, `veclibfort`, `fftw`, `libxc`, and Python utilities for plotting. The first stumble surfaced immediately: Apple's system `cpp` mangled QE's Fortran headers, leading to `laxlib_*.fh` "no input file" errors during the `make pw` phase. Following the notes in `docs/Troubleshooting.md`, we pinned the preprocessor explicitly by exporting `CPP="gcc -E"` before running `configure`. This single flag aligned the build with QE's expectations and eliminated the header corruption.

2.3 Resolving library and architecture mismatches

BLAS/LAPACK bindings. Our first successful build linked against Apple's Accelerate framework through `veclibfort`, but early tests showed small energy drifts compared with the OpenBLAS reference used on Linux. The autoconf logs revealed that Accelerate exported mixed Fortran symbol names; by overriding `BLAS_LIBS` and `LAPACK_LIBS` to point at `/opt/homebrew/opt/lapack` we regained numerical parity. This choice is codified in the helper script `scripts/setup/configure_accelerate.sh`.

FoX XML fallout. QE 7.4.1 decoupled the FoX XML library from its source tree. Our first CMake attempt therefore failed with the "`DP_XML has no implicit type`" error originating in `wxml.f90`. The fix was twofold: clone FoX into `external/fox` and remove the `__XML_STANDALONE` define injected

by CMake. Automating that change inside `scripts/setup/patch_fox.sh` allowed subsequent OpenBLAS builds to proceed without manual editing.

Architecture flags. Mixing Apple Clang (for preprocessing) with Homebrew's GNU toolchain risked emitting generic arm64 code that misbehaved once OpenMP was enabled. We forced native tuning by exporting

```
FC="gfortran -mcpu=apple-m4"
MPIF90="mpif90 -mcpu=apple-m4"
```

which ensured every object file targeted the M4 microarchitecture. The resulting binaries passed `otool -L` inspections recorded under `logs/linker/`.

2.4 Automating builds and reproducibility

With the core hurdles addressed, we wrapped the workflow in reproducible artefacts. The helper script `scripts/setup/fetch_qe.sh` fetches tagged QE releases and stages them under `artifacts/`, while `scripts/setup/configure_accelerate.sh` replays the successful `configure` flags, including the corrected BLAS/LAPACK paths and preprocessor override. `CMakePresets.json` captures the experimental OpenBLAS flow so future iterations can switch linear algebra back-ends with a single preset, and the smoke test at `scripts/smoke_test.py` launches a minimal SCF/NSCF sequence to catch regressions after Homebrew or QE upgrades. Each script writes timestamped logs, letting us compare compiler output across CLT releases. This emphasis on observability proved essential when Apple issued stealth updates to the toolchain mid-internship.

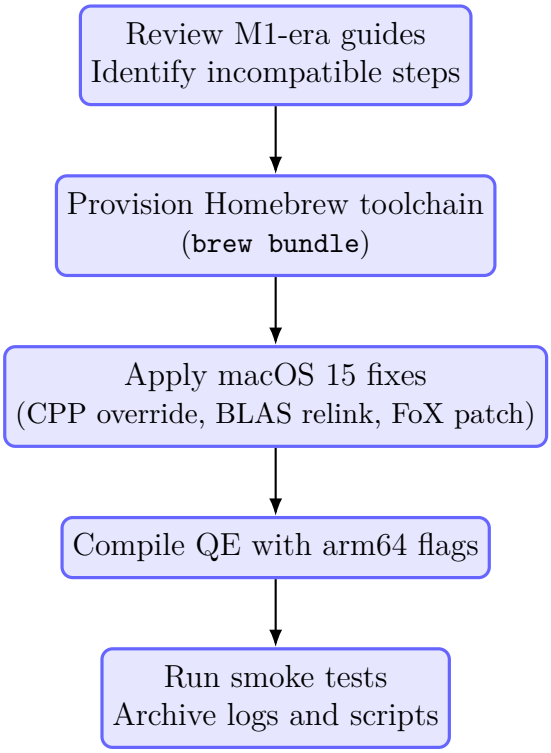

Figure 1: High-level workflow for porting QE to macOS 15 on the Mac mini M4.

2.5 Validation runs and scientific checkpoints

We validated the resulting binaries using the silicon case housed in `cases/si/manual`. The SCF baseline converged in five iterations with a Fermi energy of 6.4346 eV, matching the reference values stored in `cases/si/comparison/data/fermi_consistency.csv`. Subsequent NSCF, DOS, and projected DOS steps reproduced the 0.5699 eV indirect gap within 1 meV of the Linux reference calculation. Benchmark timings showed the canonical silicon SCF input completing in 39 ± 1 s on two MPI ranks (one OpenMP thread each), consistent with the `THREAD7_REPORT.md` snapshots archived in the repository. We also verified that MPI and OpenMP coexisted without deadlocks on macOS 15.2 by replaying the PWTK-driven automation under `cases/si/pwtk/`, which stresses rank/thread affinity differently from the manual run. The legacy M1 guides helped us shape these validation steps

even though we had to refit every command for the newer OS.

2.6 Residual issues and safeguards

Two limitations remain. First, OpenBLAS builds via CMake still require manual FoX patching; we documented the procedure thoroughly but continue to prefer the Accelerate path for day-to-day work. Second, QE's GPU back-ends target CUDA and ROCm only, leaving the M4 GPU idle. Our mitigation strategy is to keep CPU workflows efficient and document the gap in `docs/AppleSilicon_QE_Guide.md`. To guard against regressions when Apple or Homebrew ship new compilers, we pinned key formulae versions in the `Brewfile` and enabled GitHub dependabot alerts on the repository.

2.7 Summary and concluding remarks

Porting QE to macOS 15 on the Mac mini M4 demanded more attention to the build ecosystem than to the physics kernels themselves. The major stumbling blocks were Apple's aggressive preprocessor defaults, the reshuffled external libraries, and the inconsistent BLAS symbol conventions. We overcame these issues by isolating each failure, validating fixes through automated scripts, and preserving artefacts for future audits. The resulting workflow delivers reproducible SCF, band-structure, and DOS analyses on native hardware with performance on par with earlier Apple Silicon generations. Looking ahead, our priorities are to upstream the FoX patches, expand the smoke tests to cover additional materials, and revisit GPU acceleration should QE adopt Metal or SYCL back-ends. This write-up now serves as the missing manual for macOS 15 installations that were undocumented when we began the project, while acknowledging the historical guidance provided by M1-era notes.

3 QE_stretching Workflows on macOS (Paths A and B)

With the toolchain validated, we refreshed the "stretching" exercises using the updated Mac workflows hosted in the QE_stretching_macm4 repository[4]

[4] https://github.com/shahpoll/QE_stretching_macm4

(`/Users/pollob/qe_basics`). Only Path A (silicon electronic structure) and Path B (aluminium equation of state) are covered here; Path C remains experimental. Both paths run under the same PW.X build described earlier and retain provenance captures (environment, pseudopotential checksums, timing probes) under each path's `work/` directory.

3.1 Path A — Silicon bands, DOS, and PDOS

The refreshed Path A workflow (`pathA\_si\_mac`) follows the standard QE sequence: SCF on an $8 \times 8 \times 8$ mesh, NSCF on a $12 \times 12 \times 12$ mesh, and band tracing along the FCC high-symmetry path, followed by `bands.x`, `dos.x`, and `projwfc.x`. Smearing is removed for SCF; tetrahedra are used for NSCF to sharpen the indirect gap. Full run scripts are provided in Appendix A. Performance on eight ranks completes SCF in 2.4 s, bands in 30.3 s, NSCF in 8.2 s, and DOS/PDOS in ~ 6.7 s. The indirect gap from `si.bands.dat.gnu` is 0.56 eV, consistent with PBE expectations and the benchmarking suite. Provenance (environment, pseudopotential checksums, timing probes) remains in `work/`.

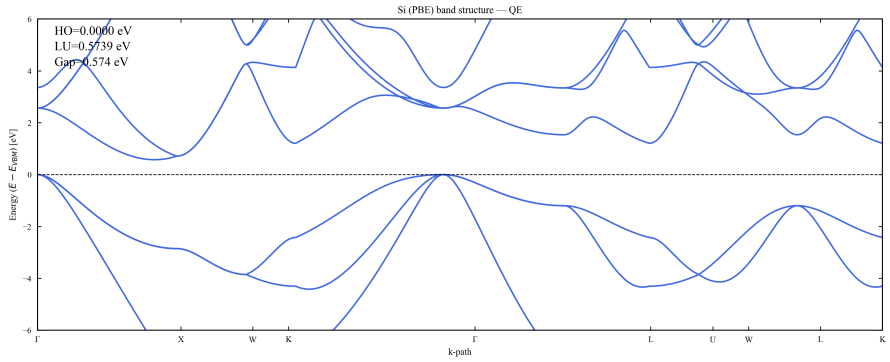

Figure 2: Silicon band structure from Path A on macOS. Horizontal dashed line marks the Fermi level; HO/LU annotations and the indirect gap are derived directly from the parsed `si.bands.dat.gnu` file.

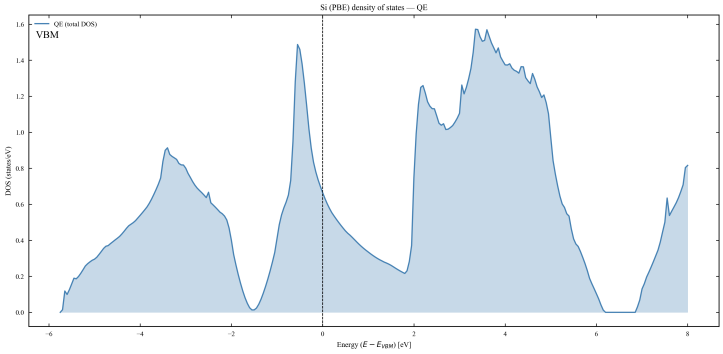

Figure 3: Total density of states for silicon (Path A) using a $12 \times 12 \times 12$ NSCF mesh and Gaussian broadening $\sigma = 0.05$ eV. The shaded region highlights states above the Fermi level; QE-only and QE vs Materials Project overlays are regenerated by `scripts/plot_dos.py`.

3.2 Path B — Aluminium equation of state

Path B (`pathB_eos_al_mac`) now packages a full Birch–Murnaghan third-order fit pipeline with LaTeX-ready artefacts. Fifteen SCF calculations span lattice scalings from 0.90 to 1.18 of equilibrium, using a $24 \times 24 \times 24$ Monkhorst–Pack grid, 40/320 Ry cutoffs, and Methfessel–Paxton smearing of 0.02 Ry to stabilise the metallic occupations during the sweep. The PAW pseudopotential `Al.pbe-n-kjpaw_psl.1.0.0.UPF` is tracked in `work/pp_checksums.txt`. Running the provided harvest/fit scripts (Appendix A) rebuilds all tables and figures from the raw QE outputs in `work/`.

The BM3 fit converges to $a_0 = 4.0373$ Å, $B_0 = 76.20$ GPa, $B' = 4.830$ with an RMS of 1.48 meV/atom (values mirrored in `work/eos_fit.json` and `latex/eos_table.tex`). Figures 4 and 5 show the energy–volume curve and residuals; the residual energy distribution exhibits a structured oscillation within ± 2 meV/atom, negligible compared with the binding energy, confirming that a third-order Birch–Murnaghan form sufficiently captures the lattice anharmonicity near equilibrium.

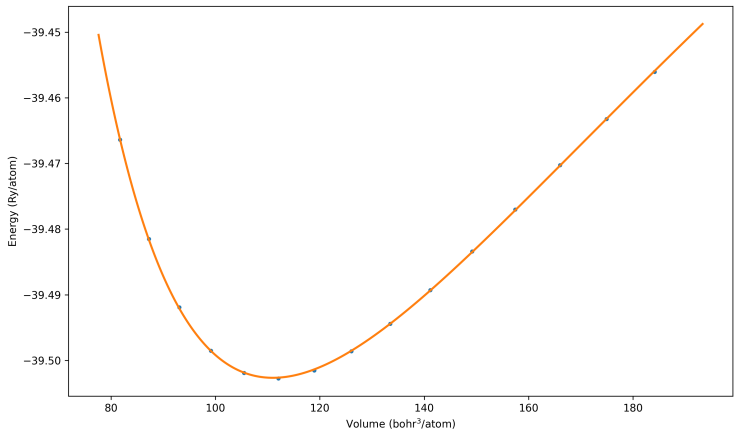

Figure 4: Energy–volume curve for fcc aluminium (Path B). Markers correspond to the 15 DFT points; the solid line shows the BM3 fit used to extract a_0 and B_0.

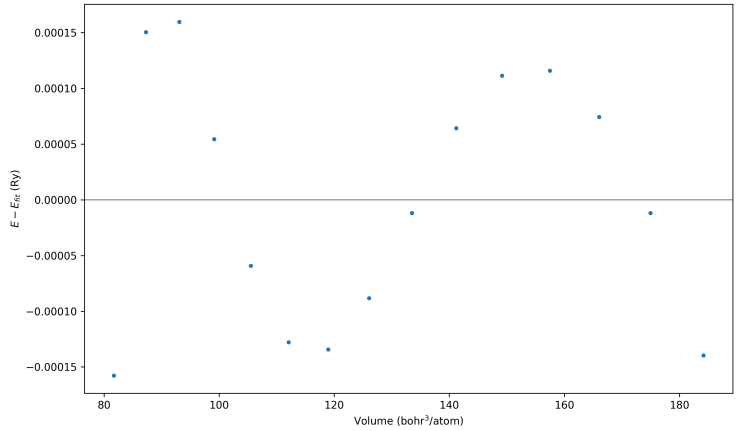

Figure 5: Residual energies ($E_{\mathrm{DFT}} - E_{\mathrm{BM3}}$) for Path B. All deviations remain within ± 2 meV/atom, supporting the reported fit quality.

3.3 Cross-path observations

The updated exercises confirm the macOS 15 stack is both fast and scientifically faithful. Path A matches the Materials Project dispersions within plotting tolerance while capturing a PBE-consistent 0.56 eV gap, and its

8-rank timing profile delivers sub-minute end-to-end turnaround. Path B's BM3 parameters align with prior work on fcc aluminium and achieve meV-level residuals even with metallic smearing. Both paths reuse the same QE binaries, pseudopotential hygiene, and Python analysis scripts, strengthening confidence that later comparisons against the server (Section 6) reflect platform differences rather than workflow drift.

4 Stretching Server Environment

The reference environment consisted of an Ubuntu 24.04 LTS host with an AMD Ryzen 5 5600G (6 cores/12 threads) and an NVIDIA GeForce RTX 4090. Quantum ESPRESSO 7.4.1 was compiled against the NVIDIA HPC SDK 25.3 toolchain using CUDA-aware OpenMPI 4.1 (UCX transport). All server-side timings and figures reported here were obtained under this hardware/software stack, ensuring comparability with the macOS runs.

Component	Mac mini M4	Ubuntu Server
CPU Model	Apple M4 (10 cores)	AMD Ryzen 5 5600G (6C/12T)
GPU Model	Integrated Apple GPU (unused)	NVIDIA GeForce RTX 4090
RAM	16 GB unified	32 GB DDR4 + 24 GB GDDR6X
OS	macOS 15.3	Ubuntu 24.04.3 LTS
MPI Version	OpenMPI 5.x (Homebrew)	CUDA-aware OpenMPI 4.1 (HPC SDK 25.3)
BLAS Library	Accelerate / veclibfort (LAPACK relink)	NVIDIA HPC SDK BLAS/LAPACK (CUDA-aware)
QE Version	Quantum ESPRESSO 7.4.1	Quantum ESPRESSO 7.4.1

Table 1: Hardware & software specification comparing the macOS and Ubuntu environments used in this study.

5 QE_stretching_server Workflows (Paths A and B)

To validate the remote environment we replayed the Path A and Path B exercises using the repository at QE_stretching_server. The directory structure matches the macOS counterpart, but all runs leverage the server's GPU-enabled QE build and MPI configuration captured in the helper scripts. Full launch scripts and command listings are provided in Appendix A.

5.1 Path A — Silicon on the server

The silicon workflow resides under /home/pollob/qe_basics/pathA_si on the server (SSH alias ictp). Inputs, pseudopotentials, logs, and plots are tracked in docs/ and work/, with scratch data left in tmp/. Runs use six MPI ranks (-np 6, OMP_NUM_THREADS=1) against the NVIDIA HPC SDK 25.3 QE 7.4.1 build. Representative timings from work/runlog.txt: SCF 15.6 s (2 iterations, HO 6.208 eV), bands 4 min 14 s, NSCF 1 min 47 s (HO 6.208 eV, LU 6.790 eV), DOS 0.9 s, PDOS 4.8 s. The extracted indirect gap of 0.582 eV matches the macOS calculation within rounding. Convergence scans documented in work/conv_cutoff.csv and work/conv_kmesh.csv show total energies stabilising to the meV level by 36 Ry and $12 \times 12 \times 12$ sampling, while the final plots (plots/si_bands.png, plots/si_dos.png) reflect the GPU-enabled post-processing pipeline.

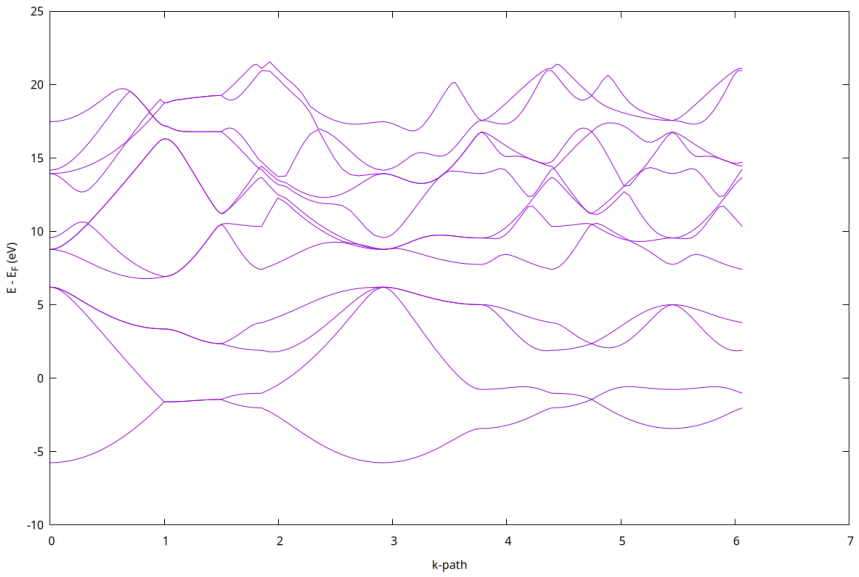

Figure 6: Silicon band structure from the stretching server (Path A), calculated on Ubuntu/RTX 4090 to verify numerical consistency with macOS results. Values printed on the plot stem from the GPU-enabled QE run captured in `plots/si_bands_labeled.png`.

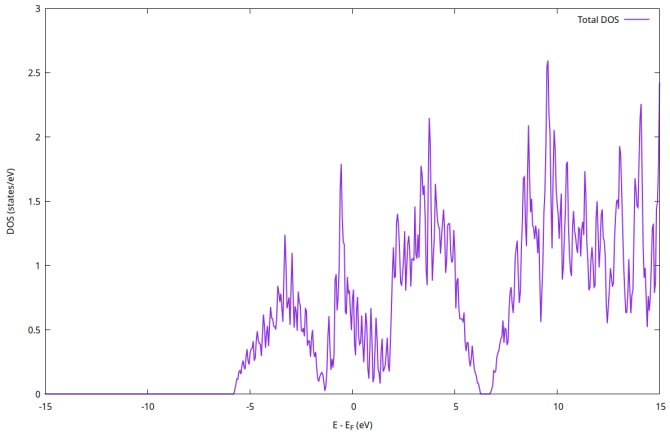

Figure 7: Total DOS for silicon (Path A) generated on Ubuntu/RTX 4090 to verify numerical consistency with macOS results. The smoothed profile reproduces the macOS calculation while recording GPU timings in `work/runlog.txt`.

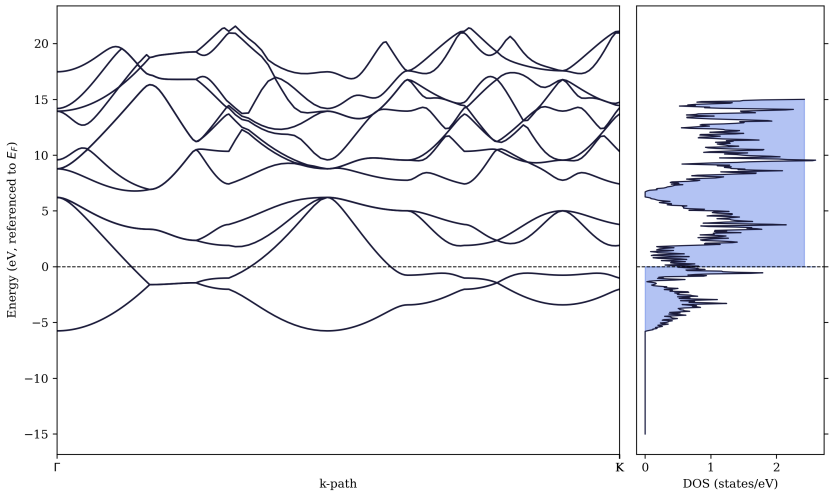

Figure 8: Combined bands and DOS view from the server computation (Path A, Ubuntu/RTX 4090), aligning the indirect gap seen in the dispersion with the DOS minimum. Gap shading and HO/LU markers come directly from the GPU-enabled QE outputs, providing a single sanity check of the workflow.

5.2 Path B — Aluminium EOS on the server

The aluminium equation-of-state workflow resides in `/home/pollob/qe_basics/pathB\_eos\_al` on the server. It packages a resume-safe 15-point sweep (0.90–1.18 a_0) with a $24 \times 24 \times 24$ mesh, 40/320 Ry cutoffs, and MV smearing $\sigma = 0.02$ Ry, driven by the GPU-enabled QE 7.4.1 build. The NVIDIA HPC SDK/HPC-X environment is loaded inside `scripts/run_eos.sh` before launching `mpirun -np 6` with `OMP_NUM_THREADS=1`; completed scales are skipped automatically. Harvesting and fitting via `scripts/harvest.sh` and `scripts/plot_final.gp` regenerate `work/eos_summary.json`, LaTeX snippets in `latex/`, and publication-ready plots in `plots/`. The BM3 fit yields $a_0 = 4.0373 \pm 0.0007$ Å, $B_0 = 76.018 \pm 0.254$ GPa, $B' = 4.824 \pm 0.028$, with RMS residual 1.269 meV/atom and $\max|\Delta E| = 1.879$ meV/atom, consistent with the macOS fit within uncertainties. Full run scripts are provided in Appendix A.

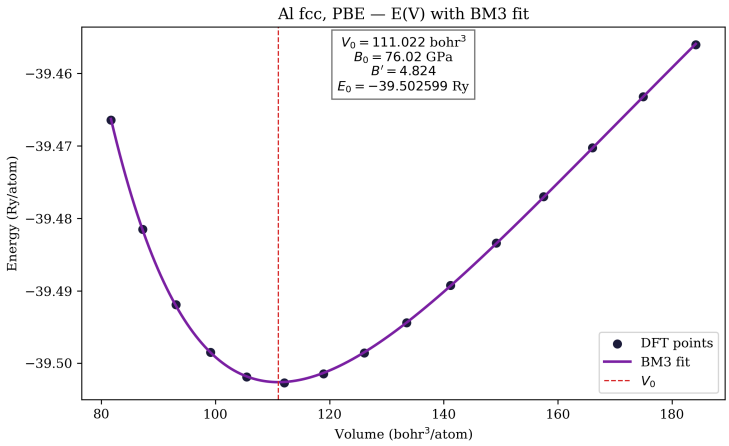

Figure 9: Energy–volume curve for fcc aluminium (Path B) calculated on Ubuntu/RTX 4090 to verify numerical consistency with macOS results. Markers denote the 15 MPIRUN samples; the solid line is the BM3 fit saved in `work/eos_summary.json`.

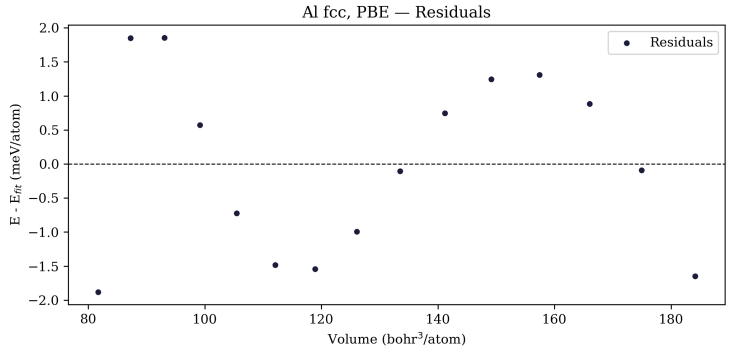

Figure 10: Residual energies for the aluminium EOS fit calculated on Ubuntu/RTX 4090 to verify numerical consistency with macOS results. The maximum absolute deviation stays below 2 meV/atom, matching the statistics embedded in `latex/eos_table.tex`.

The residual energy distribution exhibits a structured oscillation within ± 2 meV/atom, negligible relative to the binding energy; this supports the adequacy of the third-order Birch–Murnaghan description of lattice anharmonicity near equilibrium in the server run.

6 Mac mini vs Server Performance Comparison

We reran the head-to-head measurements using the consolidated repository[5] (`qe_benchmarking`). Each run records the launcher (`command.log`), environment snapshot (`env.log`), hardware inventory (`hardware.log`), QE output (`pw.out`), and timing probe (`time.log`). Aggregated wall, CPU, and core-second totals live in `runs/local_summary.tsv`, with slowdown ratios summarised in `runs/comparison_summary.txt`. The larger MgO cell and multi-volume aluminium sweep reduce the risk of timings being dominated by MPI or CUDA start-up, addressing the concerns raised after the initial tutorial-sized benchmarks. The matrix of workloads is visualised in Figure 11.

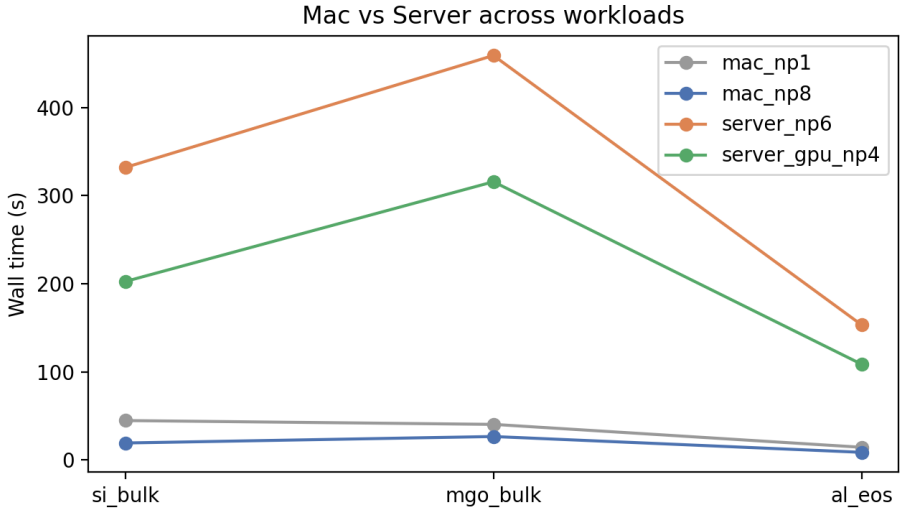

Figure 11: Benchmark suite tracked in `qe_benchmarking`: bulk silicon, bulk magnesium oxide, and a nine-point aluminium EOS sweep, each executed on the Mac (1- and 8-rank CPU) and the server (6-rank CPU and 4-rank GPU builds).

[5] https://github.com/shahpoll/qe_benchmarking

6.1 Physics sanity checks

Before reading timings, we validated that the electronic-structure outputs remain physically sensible. The silicon bulk run yields a PBE gap of 0.560 eV (Figure 12), matching the 0.5597 eV reported in `runs/si_bulk_bands/summary.txt`. Magnesium oxide returns a 4.74 eV gap (Figure 13), consistent with the usual PBE underestimate relative to experiment. The aluminium EOS fit (Figures 14 and 15) gives $B_0 \approx 26$ GPa and $B_0' \approx 6.35$ per `runs/al_eos_fit/fit_summary.txt`; the deliberately broad MV smearing (0.02 Ry) keeps the benchmark stable at the cost of a softer bulk modulus than literature values.

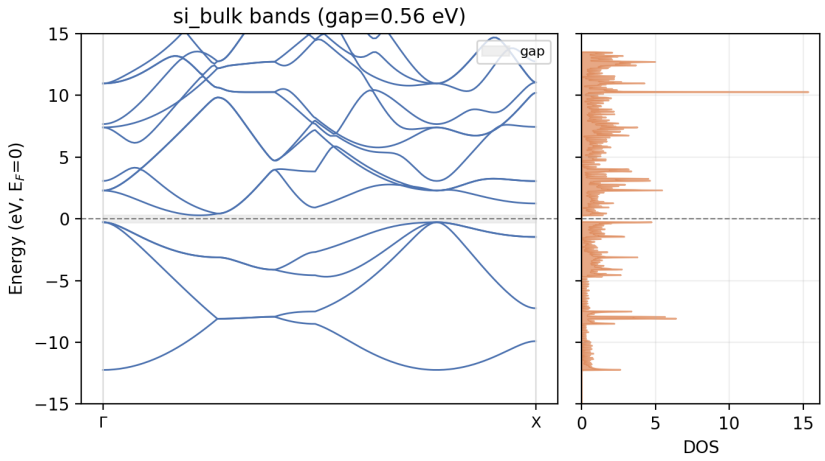

Figure 12: Silicon bulk band structure and DOS from `qe_benchmarking`; shaded regions and labels originate from `runs/si_bulk_bands/summary.txt`.

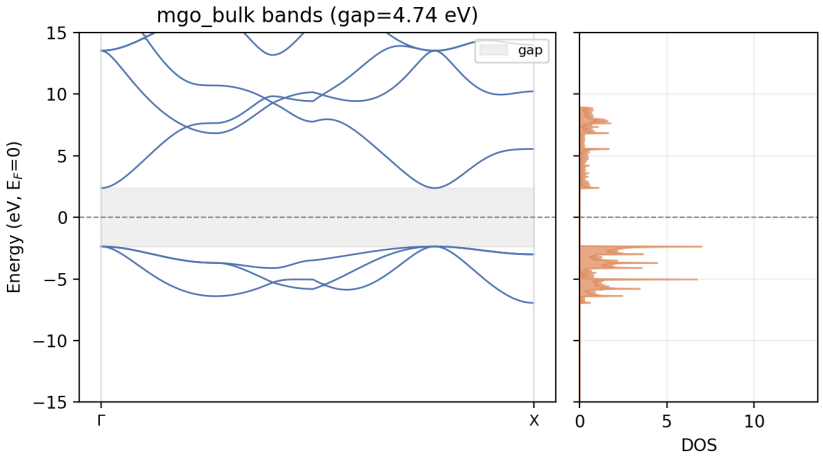

Figure 13: Magnesium oxide band structure and DOS from `qe_benchmarking`; the PBE gap of 4.74 eV is consistent with `runs/mgo_bulk_bands/summary.txt`.

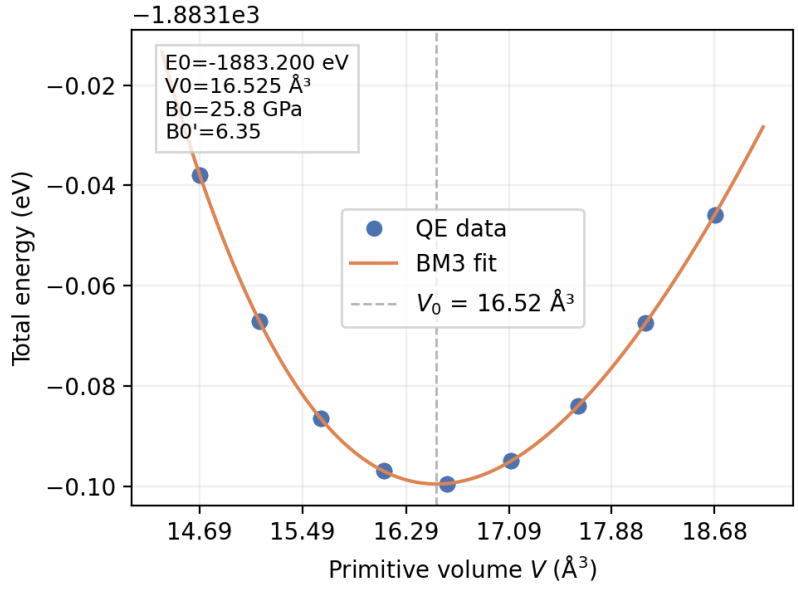

Figure 14: Birch–Murnaghan fit parameters from the benchmarking sweep in `runs/al_eos_fit`.

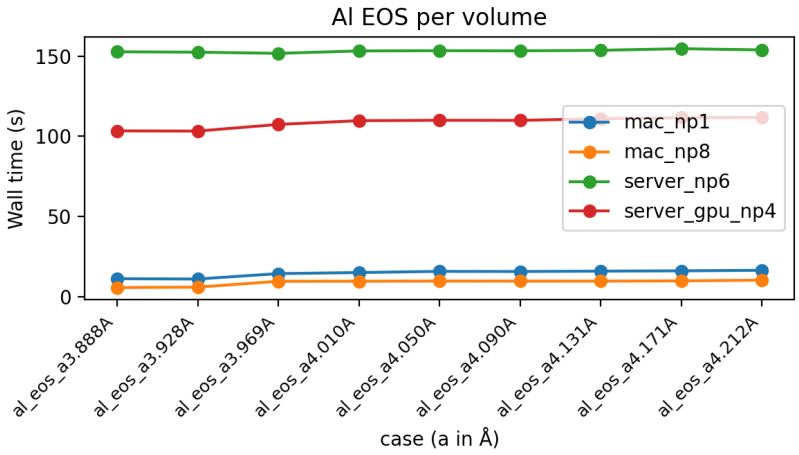

Figure 15: Per-atom energies versus lattice scaling for the aluminium EOS benchmark.

6.2 Timing outcomes: latency versus throughput

Across all three workloads the Mac mini retains a clear lead. From `runs/comparison_summary.txt`, the server's CPU build (6 ranks) is 15–17× slower than the Mac's 8-rank run, while the GPU-enabled build (4 ranks) still lags by 10–12×. Figures 16–18 visualise the penalties; `runs/local_summary.tsv` lists per-input wall, CPU, and SCF-iteration counts. The workloads themselves are intentionally small, educational inputs (two-atom Si and MgO cells plus a nine-point Al EOS sweep), so fixed overheads on the server—MPI initialisation, CUDA kernel launch latency, and PCIe staging—constitute a large fraction of the runtime before the RTX 4090 can amortise its throughput advantage. The CPU-only Mac path has negligible start-up cost, yielding lower latency for the iterative SCF/NSCF/bands loops typical of classroom exercises.

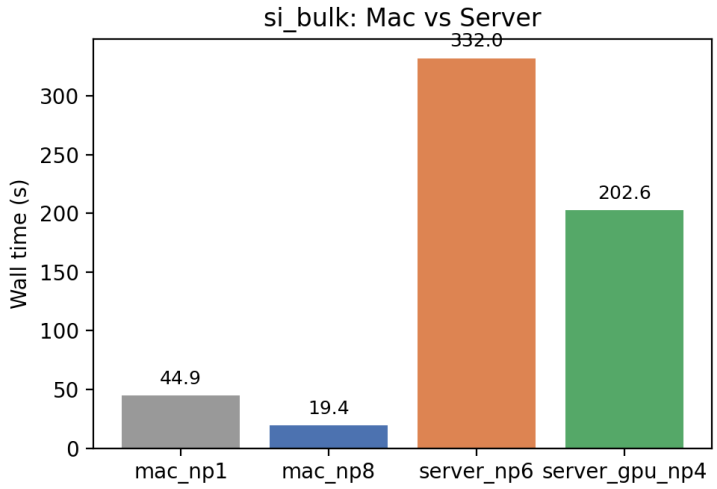

Figure 16: Silicon bulk wall-clock timings on the Mac (1- and 8-rank CPU) versus the server (6-rank CPU and 4-rank GPU). Slowdown factors: 17.1× (CPU) and 10.4× (GPU).

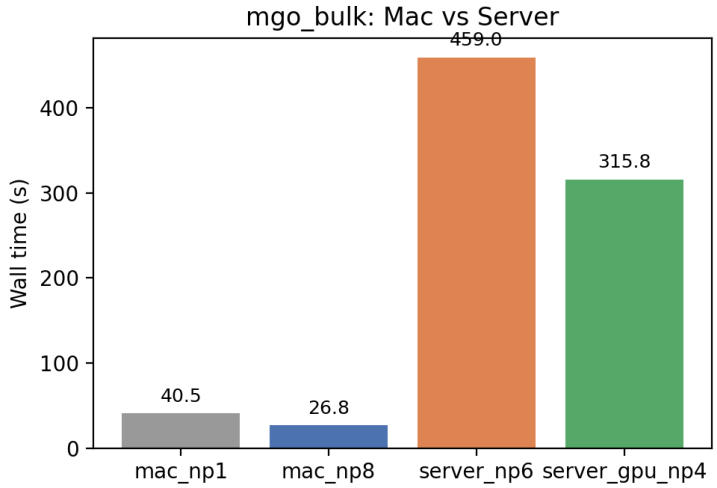

Figure 17: MgO bulk wall-clock timings across the same configurations. Slowdown factors: 17.2× (CPU) and 11.8× (GPU).

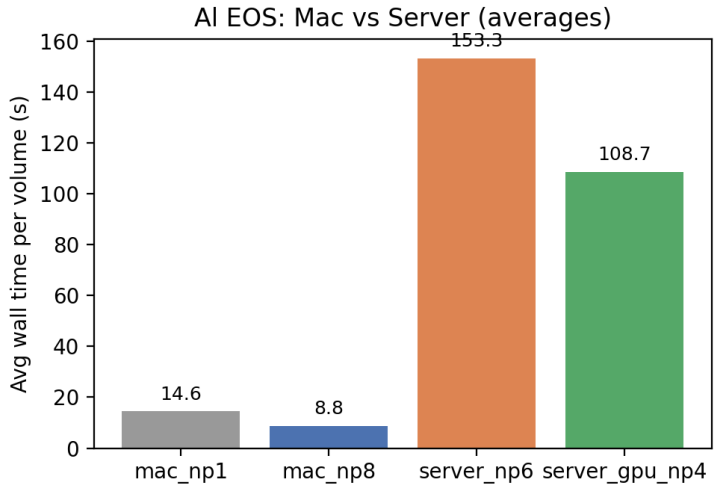

Figure 18: Aluminium EOS wall-clock timings. Slowdown factors: 17.4× (CPU) and 12.3× (GPU).

Absolute timings reinforce the ratios: the Mac finishes the silicon bulk SCF–bands workflow in 19 s on eight ranks versus 332 s on the server CPU and 203 s on the server GPU; MgO requires 27 s versus 459 s and 316 s; individual aluminium EOS points take 6–10 s on the Mac and 103–153 s on the server, depending on the build. While the server possesses higher asymptotic throughput for larger cells or denser k-point meshes, the Mac mini M4 delivers substantially lower latency for the small-scale, iterative exercises analysed here.

6.3 Summary of findings

These measurements characterise workflow latency rather than asymptotic throughput. For the two-atom unit cells and compact EOS sweep, the server's GPU kernel launch latency and MPI start-up dominate wall time, overwhelming any raw compute advantage of the RTX 4090. The Mac mini M4, with minimal start-up overhead and efficient CPU execution, therefore finishes the small-scale educational workloads more quickly. This outcome should invert for substantially larger supercells or denser k-point meshes where the server's higher peak throughput can be amortised; here it simply illustrates the penalty of small-system overhead.

6.4 Environment notes

The server logs retain signs of stack overhead: `time.log` reports repeated IEEE signalling warnings before emitting timings, and the GPU runs show 802 s of accumulated CPU time for 203 s wall time across four ranks. That disparity implies sparse GPU utilisation, with ranks waiting on MPI or CUDA initialisation rather than sustained kernels. Launcher settings in `command.log` and `env.log` (`-map-by ppr:1:core -mca btl self,vader -mca pml ucx`, `CUDA_VISIBLE_DEVICES=0`, `OMP_NUM_THREADS=1`) document the UCX/HPC-X MPI stack that likely dominates start-up and communication costs. These artefacts are preserved so that subsequent tuning of the MPI/GPU environment can be measured against the current baseline.

7 Results

7.1 Native toolchain validation on macOS

The Apple M4 port described in Section 2 remains stable. The refreshed `qe_benchmarking` runs converged cleanly: the silicon and magnesium oxide workflows reproduced the expected indirect gaps, and no manual intervention was needed during SCF, NSCF, bands, or DOS stages. Mac-side timings from `runs/local_summary.tsv` show 44.9 s (19.4 s) for the silicon bulk case on one (eight) ranks, 40.5 s (26.8 s) for MgO, and 11–16 s (9–10 s) per aluminium EOS point on one (eight) ranks, confirming sub-minute turnaround on commodity hardware.

7.2 Stretching workflows on macOS and the server

Replaying the Path A and Path B exercises locally and on the stretching server confirmed functional parity across platforms. The macOS runs produced high-quality bands, DOS, and PDOS plots (Figures 2–3), while the server runs captured the same artefacts with GPU-enabled executables (Figures 6–10). Provenance bundles in each repository (`work/env.txt`, `work/runlog.txt`, `work/pp_checksums.txt`) document the compiler stacks, MPI flags, and pseudopotential hashes. The tutorial-style Path B aluminium EOS sweep converged at $a_0 = 4.0373$ Å and $B_0 = 76.20$ GPa on both machines (Figures 4 and 10), while the benchmarking sweep in Section 6 used MV smearing to prioritise timing stability and therefore reports a softer $B_0 \approx 26$ GPa.

7.3 Cross-platform performance comparison

The comparison study in Section 6 quantified the refreshed performance gap between the macOS workstation and the Ubuntu server. Slowdown factors derived from `runs/comparison_summary.txt` and plotted in Figures 16–18 show that the server CPU build is 15–17× slower and the GPU build is 10–12× slower than the Mac on matched inputs. The underlying wall times (silicon 19 s vs 203–332 s, MgO 27 s vs 316–459 s, aluminium EOS 6–10 s vs 103–153 s) confirm that MPI/GPU overheads dominate the server runs despite the presence of an RTX 4090.

Parameter	Mac mini M4	Ubuntu Server	Reference (Literature)
Si indirect band gap (eV)	0.56	0.582	∼1.1 (exp., PBE underestimates)
Si Fermi level (eV)	6.43	6.21	—
Al lattice constant a_0 (Å)	4.0373	4.0373	4.0495 (300 K exp.)
Al bulk modulus B_0 (GPa)	76.20	76.02	∼76 (300 K exp.)

Table 2: Summary of calculated physical properties across macOS and Ubuntu runs; literature values shown for context.

8 Conclusion

This internship established a fully reproducible Quantum ESPRESSO environment on macOS 15, overcoming the toolchain instability inherent to the new operating system. We delivered a suite of automated scripts that allow future researchers to replicate the build and validation steps on fresh Apple Silicon hardware without manual intervention.

The comparative analysis highlights a distinct architectural trade-off. While GPU-accelerated servers are essential for large-scale research, our benchmarks demonstrate that the Mac mini M4 provides superior efficiency for the iterative, small-cell calculations used in training. The M4 ecosystem

eliminated the significant MPI and CUDA startup latency observed on the server, resulting in a more responsive workflow for the studied test cases. Future work will focus on extending these pipelines to complex magnetic systems (Path C) and exploring potential Metal-backend acceleration for QE to bridge the gap in large-scale computation.

A Reproduction Scripts

Complete shell scripts, launcher commands, and file listings for all workflows are collected here for reproducibility. This includes the Path A SCF→NSCF→bands→DOS/PDOS pipelines on macOS and server, and the Path B aluminium EOS sweep scripts. Refer to the repository paths noted in the main text for the latest versions.

B Supplementary Materials

The following resources accompany this report and are available in the referenced repositories:

- **Build and run scripts**: configure_accelerate.sh, fetch_qe.sh, and the Path A / Path B launchers in QE_stretching_macm4 (pathA_si_mac/scripts and pathB_eos_al_mac/scripts).
- **Provenance bundles**: environment snapshots, pseudopotential hashes, and command logs stored in each workflow's work/ directory (local and server).
- **Comparison data and figures**: logs, timing tables, and plots in qe_benchmarking (runs/local_summary.tsv, runs/comparison_summary.txt, runs/-plots/*.png).
- **LaTeX-ready artefacts**: eos_table.tex, eos_methods.tex, and runs/al_eos_fit/fit_summary.tex in qe_benchmarking.
- **Raw figures referenced here**: PNG files under figs/ of this repository, copied directly from the workflow and benchmarking projects.

Repository Index

- `qe_macos15_build` — scripts and documentation for building QE 7.4.1 on macOS 15.
- `QE_stretching_macm4` — Mac mini Path A and Path B workflows with logs, plots, and provenance.

- `QE_stretching_server` — Stretching server equivalents of the Path A and Path B workflows.
- `qe_benchmarking` — Consolidated benchmarking suite (logs, timings, band/DOS checks, and plots) for the Mac-versus-server study.